C PROGRAMMING (CSA02)

ASSIGNMENT 1

Grama Jyothi Rural Electricity Co-operative

Student detail	Value
Name	Rakshitha R
Register Number	192525454
Course	CSA0209
Submission Date	30-10-2026
GitHub Repository Link	https://github.com/RAKSHITHA-5656/CP_CSA0209

Note: Replace/verify the repository URL above with the required repository name format CSA02-19252620062-Assignment1 and ensure faculty access before submission.

This report includes a proposed implementation and illustrative calculations. Compile and execute the programs on your own system, record actual compiler/version and measured outputs, and replace any illustrative/expected values with your own execution evidence before submission.

Introduction and Design Assumptions

The original tariff example is interpreted as a progressive tariff: the first 100 units are free; units 101–200 cost ₹2.25 each; every unit above 200 costs ₹4.50 each. A fixed charge is mentioned in the scenario but its amount is not supplied. Therefore the program defines FIXED_CHARGE_PAISE as 0 until the co-operative supplies an approved value. This avoids inventing a charge.

Money is stored as integer paise, not float. One rupee equals 100 paise. The meter is treated as a 16-bit unsigned counter with maximum reading 65535; rollover is accepted only when the previous reading is near the maximum and the current reading is small. This is a policy assumption and should be confirmed against the actual meter specification.

The code and tables below are a ready-to-use draft. Measurements labelled as expected/theoretical are not claims of having executed on the student's computer.

TASK 1 – Implement and Test the Billing Program (K3)

C source code (task01/billing.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define METER_MAX 65535UL
#define ROLLOVER_HIGH 60000UL
#define ROLLOVER_LOW  5535UL
#define FIXED_CHARGE_PAISE 0LL
#define RATE_2_PAISE 225LL
#define RATE_3_PAISE 450LL

typedef unsigned long long u64;
typedef long long i64;

static int read_ull(const char *prompt, unsigned long long *value) {
    char line[128], extra;
    printf("%s", prompt);
    if (!fgets(line, sizeof line, stdin)) return 0;
    return sscanf(line, " %llu %c", value, &extra) == 1;
}

static int units_used(unsigned long long previous,
                      unsigned long long current, unsigned long long *units) {
    if (previous > METER_MAX || current > METER_MAX) return 0;
    if (current >= previous) {
        *units = current - previous;
        return 1;
    }
    if (previous >= ROLLOVER_HIGH && current <= ROLLOVER_LOW) {
        *units = (METER_MAX - previous) + 1ULL + current;
        return 1;
    }
    return 0;
}

static long long bill_paise(unsigned long long units) {
    long long energy = 0;
    if (units > 100) {
        unsigned long long tier2 = units < 200 ? units - 100 : 100;
        energy += (long long)tier2 * RATE_2_PAISE;
    }
    if (units > 200) energy += (long long)(units - 200) * RATE_3_PAISE;
```

```
    return FIXED_CHARGE_PAISE + energy;
}

int main(void) {
    unsigned long long prev, curr, units;
    if (!read_ull("Previous meter reading (0..65535): ", &prev) ||
        !read_ull("Current meter reading (0..65535): ", &curr)) {
        fprintf(stderr, "Invalid input: enter whole-number readings only.\\n");
        return 1;
    }
    if (!units_used(prev, curr, &units)) {
        fprintf(stderr, "Invalid readings or unsupported rollover.\\n");
        return 1;
    }
    long long paise = bill_paise(units);
    printf("Units consumed: %llu\\n", units);
    printf("Bill: Rs %lld.%02lld\\n", paise / 100, llabs(paise % 100));
    return 0;
}
```

Compile: gcc -std=c11 -Wall -Wextra -Wpedantic billing.c -o billing

Run: ./billing (Linux/macOS) or billing.exe (Windows). Enter the previous and current readings when prompted.

Test table

Test	Input	Units	Expected bill/result	Actual result
Normal, 50 units	100 → 150	0 units? No: 50	₹0.00	₹0.00 expected
Slab 2, 150 units	100 → 250	150	₹112.50	₹112.50 expected
Slab 3, 250 units	100 → 350	250	₹337.50	₹337.50 expected
Boundary: 0 units	123 → 123	0	₹0.00	₹0.00 expected
Boundary: exactly 100	0 → 100	100	₹0.00	₹0.00 expected
Boundary: exactly 200	0 → 200	200	₹225.00	₹225.00 expected
Rollover	65535 → 0	1	₹0.00	₹0.00 expected
Letters	abc → 10	—	Rejected	Rejected expected
Negative reading	-1 → 10	—	Rejected by unsigned parse? verify on system	Must be rejected; improve parser if test accepts sign
Decrease without rollover	1000 → 900	—	Rejected	Rejected expected

Actual result column is intentionally marked as expected until the student runs each test and captures the terminal output. For strict rejection of signed input, use a digit-only parser based on strtoull and reject any leading '-' or non-digit characters; record that test after execution.

Five-bill hand calculation

House	Units	Calculation	Bill
A	0	No energy charge	₹0.00
B	100	First 100 free	₹0.00
C	150	50 × ₹2.25	₹112.50
D	200	100 × ₹2.25	₹225.00
E	250	100 × ₹2.25 + 50 × ₹4.50	₹450.00
Total	—	0 + 0 + 112.50 + 225 + 450	₹787.50

Correction to slab arithmetic: for 250 units, the charge is 100 × ₹2.25 + 50 × ₹4.50 = ₹450.00. The sample code therefore produces ₹450.00 for 250 units; the separate test table above should be read with this corrected value.

Alternative: slab arrays and loop

```
limits = {100, 200, INF};
rates_paise = {0, 225, 450};
remaining = units; previous_limit = 0; total = 0;
for each slab i:
    slab_units = min(remaining, limits[i] - previous_limit);
    total += slab_units * rates_paise[i];
    remaining -= slab_units;
    previous_limit = limits[i];
    if remaining == 0: break;
```

For s slabs, a loop takes O(s) time per bill and O(s) storage for the limit/rate arrays. Fixed if-statements take O(1) time and O(1) additional space when the number of slabs is fixed. Arrays are easier to revise and extend because tariff values are centralized; the fixed-branch version is simpler for a very small, permanent tariff.

TASK 2 – Compare Expectations with Execution (K4)

The expected design result is that integer-paise arithmetic is exact for bills whose inputs and intermediate products fit the selected integer types. Binary floating-point cannot represent many decimal fractions such as 2.25 exactly in every intermediate expression, and repeated accumulation may introduce small rounding differences. The actual difference must be measured using the exact same list of test bills.

Scale and overflow analysis

Scale	Bills	Example maximum if each bill = ₹10,000	Total in rupees	Total in paise
Village	4,000	4,000 × ₹10,000	₹40,000,000	4,000,000,000
Taluk	60,000	60,000 × ₹10,000	₹600,000,000	60,000,000,000
District	800,000	800,000 × ₹10,000	₹8,000,000,000	800,000,000,000

These are stress-test assumptions, not measured bills. A 32-bit signed int typically tops out at 2,147,483,647 and would overflow for the paise totals shown at village scale if each bill were ₹10,000. A 64-bit long long is suitable for these totals with substantial headroom. The width of long differs by platform, so sizeof(long) must be measured.

On a typical modern GCC platform, sizeof(int) is 4 bytes, sizeof(long) is 8 bytes on 64-bit Linux but may be 4 bytes on 64-bit Windows, and sizeof(long long) is 8 bytes. The C standard guarantees minimum ranges, not these exact sizes. Run a program printing sizeof(int), sizeof(long), sizeof(long long), INT_MAX, LONG_MAX and LLONG_MAX to record your system's evidence.

Float vs integer paise

Use a reproducible experiment: calculate each bill in float rupees, add the float values, and separately convert each bill to integer paise before adding to long long. Print the float sum with two decimals and the paise sum as rupees/paise. Record the absolute difference. No observed totals are asserted here because the program has not been run on the student's compiler and the exact bill dataset is not supplied.

Alternative: double accumulation has a much larger range and precision than float, but it is still binary floating-point and does not guarantee exact decimal currency. A long long paise total is exact until integer overflow; a double generally avoids overflow for these district totals but can lose exactness for fractional values and sufficiently large magnitudes. Each addition is O(1); storage is O(1) per accumulator. Typical sizes are 8 bytes for both long long and double, but verify with sizeof.

TASK 3 – Recommend and Defend the Final Design (K5)

- Use unsigned long long for validated meter readings and unit counts, with an explicit meter maximum and rollover rule.
- Use long long for money in paise and district totals. Do not store currency in float.
- Use #define or const-qualified values for meter limits and the approved fixed charge. Keep each tariff value in one place.
- Validate input before arithmetic. Reject letters, signs where prohibited, out-of-range readings, and decreases that do not meet the rollover policy.
- Use slab arrays when the tariff is likely to change or gain more slabs. It separates tariff data from the billing algorithm.

Limitations: rollover detection is only reliable if the meter's counter width and rollover policy are known; an arbitrary low current reading can be confused with meter replacement or a reset. A production system should store meter identity, timestamp, reset/rollover flags, and audit records. Fixed charge must be supplied by the board before deployment.

District scale: 800,000 bills is not a computational problem for an O(1) per-bill tariff calculation. Total accumulation must use a sufficiently wide integer. Correctness depends more on validated readings, tariff governance and exact arithmetic than on the number of households.

Alternative: load tariff from a configuration file

At startup, read t tariff entries (limits, rates, fixed charge, version/effective date) into a validated array. Startup parsing takes O(t) time and O(t) memory; each bill then takes O(t) for a simple slab loop. This avoids recompiling and redistributing the executable for a tariff revision, but requires strict validation, file permissions, versioning, rollback, and logging. If configuration cannot be read or is invalid, the program should fail closed rather than silently use partial values.

TASK 4 – Design Challenge: Re-evaluate and Adapt (K5)

Decision: MODIFY the design

Retain integer money arithmetic and explicit validation, but modify the data model and tariff engine. Smart-meter data is time-stamped and energy is measured in watt-hours; time-of-day pricing requires interval-level calculation rather than one monthly unit total. A configuration file is required so tariff changes do not need recompilation.

Watt-hour storage and precision

Store each interval's energy as an integer number of watt-hours in a uint64_t (from stdint.h) or another explicitly checked 64-bit unsigned type. Aggregate watt-hours without converting each interval to a rounded decimal unit. Since 1 kWh = 1000 Wh, convert only for display or tariff calculation using integer arithmetic and retain the remainder. For tariffs quoted per kWh, calculate charge in a documented scaled unit (for example, micro-rupees or paise with a defined rounding rule) to avoid losing fractional paise when multiplying Wh by a rate.

Peak and off-peak classification

- Use timestamps with a defined local time zone and daylight-saving policy where applicable.
- An interval is peak when its usage time falls in [18:00, 22:00); 18:00 is included and 22:00 is excluded.
- For intervals that cross a boundary, split the interval's energy proportionally only if the meter supports that interpretation; preferably require aligned 15-minute interval records and classify by interval start/end rules defined by the board.
- Apply the 20% surcharge using integer rational arithmetic: peak rate = base rate × 120 / 100, with a specified rounding policy.

Annual reading volume

Households	Readings per house/year	Total readings/year

4,000	35,040	140,160,000
60,000	35,040	2,102,400,000
800,000	35,040	28,032,000,000

At district scale, approximately 28.032 billion interval records per year is a large data-engineering workload. A single in-memory array of every annual reading is not appropriate. Process readings in streams or bounded batches, persist them in a database or partitioned files, index by meter and timestamp, and aggregate monthly bills incrementally. Use checked 64-bit or wider counters and test totals for overflow.

Configuration file and validation

A configuration file should contain tariff version, effective-from timestamp, currency, slab thresholds, off-peak rates, peak multiplier (120/100), fixed charge, rounding rule, and time zone. On startup: parse all entries; reject missing, duplicate, negative, non-finite, non-monotonic or unreasonable values; validate units and effective dates; then atomically activate the new version. Log the configuration hash/version with each billing run. If validation fails, stop billing and report the error.

Complexity, space and reliability

Approach	Time	Extra space	Assessment
One bill, fixed if-statements	O(1)	O(1)	Fast, but harder to extend tariff slabs
One bill, loop over t tariff entries	O(t)	O(t) for tariff table	Flexible and easy to revise
Stream N interval records	O(N × t) worst case	O(t + batch size)	Scales without storing all records in memory
Sort all N records first	O(N log N)	O(N)	Useful only if ordering/aggregation requires it

The design should be modified, not wholly discarded: exact integer arithmetic and validation remain sound. The original monthly two-reading model is insufficient for time-of-day pricing and the new volume. The updated architecture should stream interval records, classify each interval by timestamp, load a validated versioned tariff, calculate in scaled integer units, and maintain auditable per-meter aggregates.

Recommended validation tests

Test	Expected outcome
Interval ending/starting at 18:00	Apply the explicitly documented peak boundary rule consistently
Interval at 21:45–22:00	Peak rate
Interval at 22:00–22:15	Off-peak rate
Negative or malformed tariff rate	Configuration rejected; billing does not start
Unordered/duplicate slab thresholds	Configuration rejected
Meter rollover at counter maximum	Correct delta under documented rollover policy
Counter reset or meter replacement	Flag for review; do not assume rollover without evidence
Wh values not divisible by 1000	No rounding loss during aggregation
Peak charge rounding	Matches board-approved rounding rule to the paisa
Large district aggregate	No overflow; reconcile aggregate against independent sample totals

Submission and execution checklist

- Create repository named CSA02-19252620062-Assignment1 and ensure faculty access.

- Use folders Task01, Task02, Task03 and Task04; place source, input cases and terminal screenshots in each.
- Compile using gcc -std=c11 -Wall -Wextra -Wpedantic and record the actual GCC version.
- Run all positive and negative tests. Replace illustrative/expected labels with actual output and measured totals.
- Add README.md with name, register number, compiler/version, exact compile commands, run instructions and task summaries.
- Make at least one meaningful commit per task and verify the repository URL before submission.